

8h às 10h

Explore os materiais e execute o notebook-base.

Autoestudo: uma trilha, não uma lista

1

Tipos de sistemas de recomendação —
visão geral (Engati)

2

Sistemas de recomendação: uma visão
aprofundada (Medium)

3

Implementação de um recomendador em
Python (DataCamp)

4

Recommender Systems in Python 101
(Kaggle)

| Stand-up daily: em pé, direto ao ponto

15 min

10:00 → 10:15

1

Em pé

A daily é rápida — todos de pé.

2

Tasks do projeto

O que cada um fez e o que falta.

3

Cronograma do dia

Definir prioridades e metas de hoje.

Objetivo: alinhar o time e destravar o dia.

DIAGRAMA DO MÓDULO



AULA 8 • SEMANA 6

Aprendizado Não Supervisionado — Parte II

Sistemas de Recomendação • User-based • Item-based • Hybrid • Elbow

Prof^a Dra. Crishna Irion

Agenda do Encontro

1

Conceito

O que é recomendar; as 3 famílias

2

Aplicação prática

O que é um sistema de recomendação
As 3 famílias: content, collaborative, hybrid
User-based e Item-based
Abordagem híbrida (hybrid)
Similaridade entre usuários e itens
Definindo o K com K-means e Elbow Plot

3

Código

Matriz usuário $\times$ item
Recomendador User-based e Item-based
Recomendação híbrida
K-means e Elbow Plot para escolher K

PARTE 1

Sistemas de Recomendação

Sugerir o item certo para a pessoa certa, a partir de padrões de comportamento.

Desafio de abertura: como a plataforma decide o que te mostrar?

```
Ana    -> curtiu: Duna, Matrix  
Bruno -> curtiu: Matrix, Blade Runner  
Ana viu Blade Runner? Ainda nao.  
Recomendar para a Ana?
```

A máquina não assistiu a nada.

Como medir que Ana e Bruno se parecem?
Recomendar pelo usuário ou pelo item?
E se um filme novo não tiver notas?

Em duplas: qual filme vocês recomendariam para a Ana — e por quê?

Três famílias de sistemas de recomendação

1

Content-based

usa atributos do item;
recomenda o parecido
com o que você gostou

2

Collaborative

usa o comportamento
de muitos usuários;
ignora atributos do item

3

Hybrid

combina as duas; cobre
as fraquezas de cada
abordagem

A maioria dos sistemas reais é híbrida.

Content-based e Collaborative resolvem o mesmo problema de formas diferentes

CONTENT-BASED

recomenda itens parecidos com os que você já gostou
usa atributos do item (gênero, autor, tags)
funciona para item novo (tem atributos)
tende a recomendar "mais do mesmo"

COLLABORATIVE FILTERING

recomenda o que usuários parecidos gostaram
usa só a matriz de avaliações (quem avaliou o quê)
descobre gostos que os atributos não capturam
sofre com o problema do "cold start"

Na prática, escolhe-se pela informação disponível — atributos ou histórico.

PARTE 2

User-based

Recomendar a partir de usuários parecidos.



User-based: pessoas parecidas gostam de coisas parecidas

a ideia

quem se parece com você?

como funciona

- 1 · achar usuários similares
- 2 · recomendar

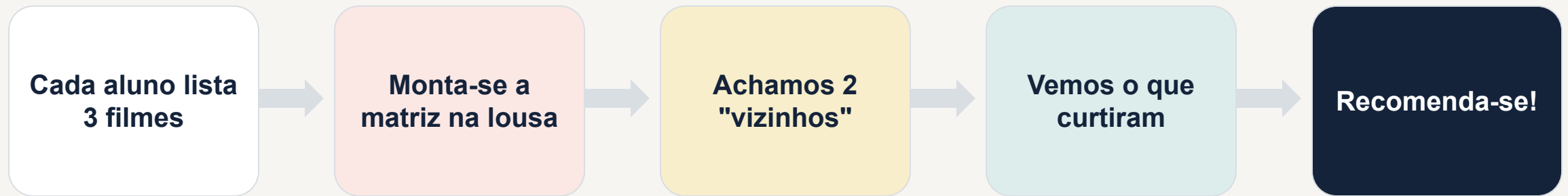
Passo a passo

Encontra os K usuários mais parecidos com o alvo (os "vizinhos"), olha o que eles avaliaram bem e que o alvo ainda não viu, e recomenda esses itens.

A semelhança vem do padrão de avaliações em comum.

Ponto fraco: recalcular vizinhos fica caro quando há muitos usuários.

Dinâmica: a turma vira uma matriz usuário × item



Sem código: só com papel e a turma, todos entendem user-based em 5 minutos.

Código: recomendador User-based



1. Montar a matriz e medir similaridade

Linhas = usuários, colunas = itens; cosseno entre usuários.

```
import pandas as pd
from sklearn.metrics.pairwise \
    import cosine_similarity
# R: usuarios x itens
sim = cosine_similarity(R)
S = pd.DataFrame(sim, index=R.index, columns=R.index)
```



2. Achar vizinhos e recomendar

Os K usuários mais parecidos indicam o que recomendar.

```
def recomendar_user(u, K=3, N=3):
    viz = (S[u].drop(u).sort_values()[-K:].index)
    notas = R.loc[viz].mean()
    ja_viu = R.loc[u] > 0
    return (notas[~ja_viu].sort_values()[-N:])
```

User-based = KNN aplicado a usuários: recomenda o que os vizinhos gostaram.

PARTE 3

Item-based

Recomendar a partir de itens parecidos aos que a pessoa já gostou.



User-based vs. Item-based

USER-BASED

"quem se parece comigo?"

similaridade entre usuários
recomenda o que vizinhos gostaram
muda sempre que as pessoas avaliam

ITEM-BASED

"o que se parece com o que gostei?"

similaridade entre itens
recomenda itens parecidos ao já curtido
mais estável e escalável na prática

Similaridade: como medir que dois itens (ou usuários) se parecem

cos θ

Similaridade do cosseno: mede o ângulo, não o tamanho.

1

Cosseno

ângulo entre dois vetores de avaliações; robusto à escala

2

Euclidiana

distância direta; sensível a quem avalia "alto" ou "baixo"

3

Pearson

correlação; desconta o viés de cada usuário

Código: matriz

1. Matriz de itens

```
import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity

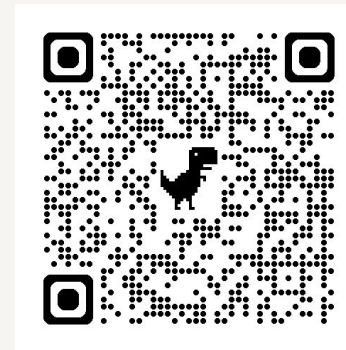
dados = {
    'Ana': [5, 5, 0, 1, 1, 0, 5],
    'Bruno': [0, 5, 5, 0, 0, 4, 4],
    'Carla': [1, 0, 0, 5, 5, 0, 0]
}

filmes =
['Duna', 'Matrix', 'BladeRunner', 'Titanic', 'Notebook', 'ET', 'Inter-
stellar']

R = pd.DataFrame(dados, index=filmes).T
R
```

Vamos construir a Matriz que montamos:

- faça uma cópia do Colab
- altere os nomes de 5 users e dos filmes que eles gostam
- Rode as recomendações



<https://encurtador.com.br/DCUR>

Código: recomendador Item-based

2. Similaridade entre itens

Transpor a matriz: agora comparamos colunas (itens).

```
sim = cosine_similarity(R)
S = pd.DataFrame(sim, index=R.index,
                  columns=R.index)
```

3. Recomendar itens parecidos

Some as similaridades com os itens que a pessoa já curtiu.

```
def recomendar_user(usuario, K=2, N=3):
    # 1) os K usuários mais parecidos (vizinhos)
    vizinhos =
S[usuario].drop(usuario).sort_values(ascending=False).head(K).index
    # 2) nota média que os vizinhos deram a cada filme
    notas_vizinhos = R.loc[vizinhos].mean()
    # 3) manter só o que o usuário-alvo ainda NÃO viu
    ja_viu = R.loc[usuario] > 0
    candidatos = notas_vizinhos[~ja_viu]
    # 4) devolver os N melhores
    return candidatos.sort_values(ascending=False).head(N)

print('Recomendações User-based para a Ana:')
recomendar_user('Ana').round(2)
```

Hybrid: combinar abordagens para cobrir as fraquezas

1

Ponderado

soma as notas de vários recomendadores com pesos

2

Alternado (switching)

troca de abordagem conforme o contexto (ex.: item novo)

3

Em cascata

um recomendador refina a lista gerada por outro

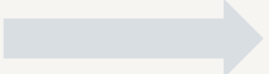
Híbrido resolve o cold start: quando falta histórico, o content-based assume.

PARTE 5

Definindo o K

Quantos grupos (ou vizinhos) usar? Deixe os dados sugerirem — com K-means e o Elbow Plot.

Elbow Plot: onde o ganho para de compensar

K = 2  **K = 4**

- rodamos o K-means para vários K
- eixo Y: inércia (soma das distâncias ao centro)
- a inércia sempre cai quando K aumenta
- o "cotovelo" é onde a queda desacelera

Escolha o K no cotovelo; confirme com a silhueta e o sentido de negócio.

Código: K-means e o Elbow Plot



1. Calcular a inércia para vários K

A inércia (WCSS) mede o quão compactos são os grupos.

```
from sklearn.cluster import KMeans
inercias = []
Ks = range(1, 9)
for k in Ks:
    km = KMeans(n_clusters=k,
                random_state=42, n_init=10)
    km.fit(X)
    inercias.append(km.inertia_)
```



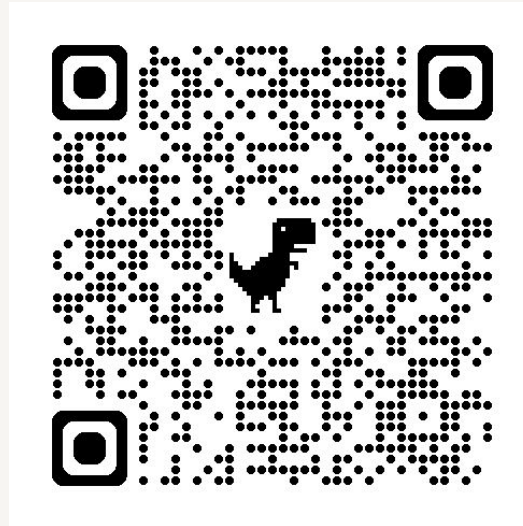
2. Traçar o gráfico e achar o cotovelo

O "cotovelo" indica o K que melhor equilibra ganho e simplicidade.

```
import matplotlib.pyplot as plt
plt.plot(list(Ks), inercias,
         'o-')
plt.xlabel('K (numero de grupos)')
plt.ylabel('Inercia (WCSS)')
plt.show()
# cotovelo ~ K=2
```

Regra: não escolha a menor inércia (K=n zera tudo) — escolha o cotovelo.

Código: Acrescentando o K-means e o Elbow Plot



<https://encurtador.com.br/xzUc>

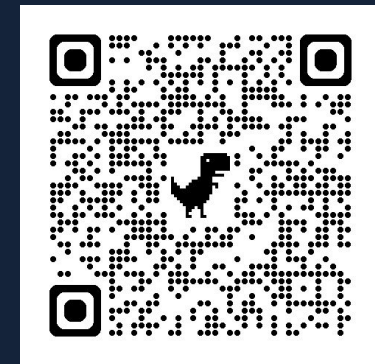
PARTE 4

Hybrid + Prática guiada



Combinar user e item-based e construir o recomendador completo no Colab.

<https://encurtador.com.br/EWKC>



Dúvidas? Vamos conversar.

Profª Dra. Crishna Irion

crishna.irion@prof.inteli.edu.br

